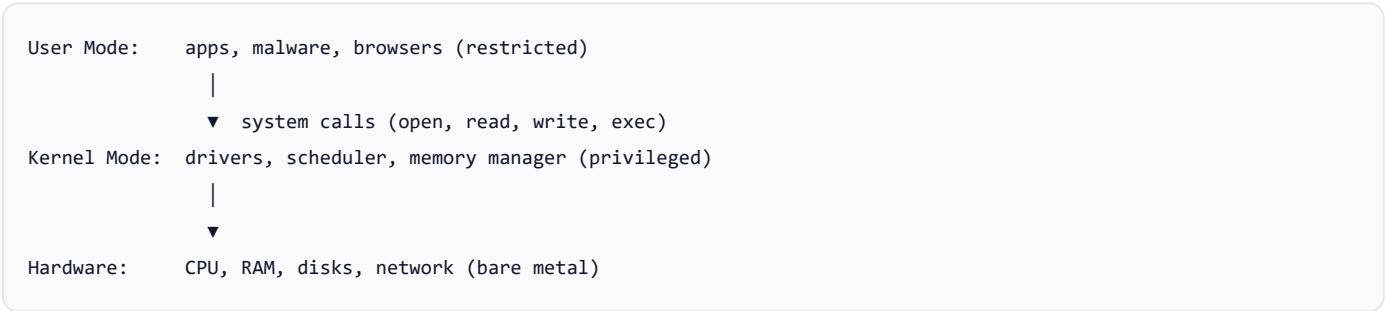


Operating Systems in Depth

The Kernel Is the Trusted Third Party

The `kernel` is the only code allowed to touch hardware directly. Everything else (your programs, malware, browsers) runs in `user mode` and must ask the kernel for every privileged operation through a `system call`.



CPU Privilege Levels (Rings)

Most CPUs provide multiple privilege levels:

Ring	Purpose
Ring 0	Kernel mode – full hardware access
Ring 1-2	Rarely used (some virtualization/device drivers)
Ring 3	User mode – restricted

Modern CPUs add `Ring -1` (hypervisor) and `Ring -2` (System Management Mode / SMM). Attacks that escalate to ring -1 or -2 are devastatingly hard to detect.

Processes and Virtual Memory

Each process gets a **virtual address space** – the illusion that it owns all of memory. The kernel maps these virtual addresses to physical RAM using **page tables**. This gives:

Isolation: Process A cannot read Process B's memory

Protection: The kernel can decide what each process may touch

Consistency: Every process sees the same layout

Security insight: A **local privilege escalation** (LPE) exploit typically chases one of these bugs:

A kernel bug reachable via an unvalidated system call

A page-table confusion that bypasses isolation

A race condition between two syscalls

System Calls as the Attack Surface

The system call interface is the largest attack surface on your machine. Hundreds of syscalls, each with parameters that must be validated. Every unvalidated parameter is a potential deserialization or buffer overflow.

Classic story: the `mmap` + syscall race used by Dirty COW (CVE-2016-5195) – a copy-on-write race let any user overwrite read-only files, including root-owned ones, and gain root.

[← CPU, Memory & the Boot Proce...](#)

[IoT & Embedded Systems Securit...](#)